

Prague University of Economics and Business
Faculty of Informatics and Statistics



Recommender system for B2B data

BACHELOR'S THESIS

Study program: Aplikovaná informatika

Field of study: Aplikovaná informatika

Author: Tomáš Zajda

Supervisor: Ing. Iva Stanovská

Prague, May 2021

Declaration

I hereby declare that I wrote this Bachelor Thesis *Recommender system for B2B data* independently and used only the sources and aids indicated.

In Prague on the 10th of May 2021

.....

Student signature

Acknowledgment

I would like to thank the supervisor of this thesis Ing. Iva Stanovská for her support, guidance, many pieces of advice, and a friendly approach.

Next, I would like to thank the company BizMachine s.r.o. that provided the input dataset. Especially to the lead data analyst Jan Fait who initially suggested the topic of the thesis, enabled the data provision, and also offered many helpful insights.

Lastly, I would like to thank my beloved family and friends for their never-ending support.

Abstrakt

Bakalářská práce se zabývá problematikou praktických doporučovacích systémů. Dokument sestává z teoretické a praktické části.

V teoretické části je nejdříve představen a definován pojem doporučovacích systémů. Poté je stručně popsána historie a vývoj těchto systémů. Následně jsou podrobně představeny základní přístupy k tvorbě takových systémů, založeny na odborných publikacích zabývajících se touto problematikou. Jsou zváženy silné a slabé stránky každého z těchto přístupů.

Jeden z přístupů popsaných v teoretické části je poté zvolen, konkrétně tzv. content-based přístup. Na základě toho přístupu je následně navržen praktický systém, který doporučuje potenciální zákazníky pro obchodníky z B2B trhu. Vstupem pro tento algoritmus jsou reálná data z českého trhu, která poskytla firma zabývající se touto problematikou.

Prvním dílčím výsledkem praktické části je automatizovaný algoritmus pro doporučování firem obchodníkům na základě jejich historických interakcí. Druhým dílčím výsledkem praktické části je webové rozhraní pro správu a testování tohoto algoritmu. Na závěr jsou zváženy silné i slabé stránky vytvořeného řešení, a jsou navrženy budoucí kroky k jeho rozšíření a vylepšení.

Klíčová slova

recommender systém, content-based filtrování, e-commerce

Abstract

This bachelors thesis examines the topic of practical recommender systems. The document consists of both theoretical and practical parts.

The theoretical part starts by describing and defining the concept of a recommender system. Then the history and development of these systems are briefly described. The main concepts and design approaches are introduced based on several publications about the topic. The strengths and weaknesses of each design approach are considered.

One of these approaches is chosen, specifically the content-based approach, with the help of the facts described in the theoretical part. The principles of this approach are used to design a practical recommender system that proposes potential customers to sales representatives in the B2B market. The input of this algorithm are real-world data from the Czech market provided by a company with experience and expertise in this field.

The first result of the practical part of the thesis is an automated algorithm that recommends companies to sales representatives based on their historical behavior. The second result is a web interface created to configure and test the algorithm. The last part of the thesis evaluates the created solution and proposes future steps to further develop and improve it.

Keywords

recommender system, content-based filtering, e-commerce

Contents

Introduction	13
1 Evolution of recommender systems	15
1.1 What is a recommender system	15
1.2 History and development	15
1.2.1 Digital bookshelf	15
1.2.2 Further development in the 1990s	16
1.2.3 The Netflix prize	16
1.2.4 Current and future research	16
2 Architecture of recommender systems	17
2.1 Structuring a recommender system	17
2.1.1 Domain	17
2.1.2 Purpose	17
2.1.3 Context	18
2.1.4 Personalization	18
2.1.5 Opinions	19
2.1.6 Privacy and trustworthiness	19
2.1.7 Interface	19
2.1.8 Algorithm	20
2.2 Chapter summary	22
3 Designing a recommender system	23
3.1 Taxonomy of the B2B recommender	23
3.1.1 Domain	23
3.1.2 Purpose	23
3.1.3 Context	24
3.1.4 Personalization	24
3.1.5 Opinions	24
3.1.6 Privacy and trustworthiness	24
3.1.7 Interfaces	24
3.2 Interface architecture	25
3.2.1 Functional requirements	25
3.2.2 Technology	25
3.2.3 Coding principles and design	25
3.3 Code style	26
3.3.1 Naming conventions	26
3.3.2 Code layout	27
3.3.3 Commenting conventions	27
3.4 Algorithm	27

3.4.1	Elements of the algorithm	28
3.4.2	Vector representation	28
3.4.3	Vector distance	28
3.4.4	User profile	29
3.4.5	Comparing data with tree structures	30
3.4.6	Randomizer	31
3.5	Chapter summary	32
4	Implementation	33
4.1	Database and dataset	33
4.1.1	Initial dataset	33
4.1.2	Database design	35
4.1.3	Structure transformation	35
4.1.4	Loading data to the database	36
4.2	Interface code structure	36
4.2.1	Website Implementation	36
4.2.2	Algorithm variables	42
4.3	Implementation of the algorithm	42
4.3.1	Vector class	42
4.3.2	Comparing NACE codes	43
4.3.3	Calculating user profile vectors	44
4.3.4	Calculating cosine similarity	45
4.3.5	Randomizer	46
4.4	Chapter summary	46
4.4.1	Calculation showcase	47
5	Evaluation	49
5.1	Algorithm	49
5.1.1	Recommender system quality metrics	49
5.1.2	Possible improvements	50
5.1.3	Comparison to other solutions	50
5.2	Interface	51
5.2.1	Possible improvements	51
	Conclusion	53
	References	55
	A Data and code	59

List of Figures

- 2.1 Comparison of collaborative and content-based systems (13) 21
- 3.1 Data flow through the MTV Django architecture (Source: Author’s own diagram) 26
- 4.1 Conceptual structure of the provided dataset (Source: Author’s own work) . . . 34
- 4.2 Physical diagram of the final database (Source: Author’s own work) 35
- 4.3 Code structure of the interface implementation (Source: Author’s own work) . . 37
- 4.4 Catalog index website with test values (Source: Author’s own work) 39
- 4.5 Settings page with engine configuration and last run log (Source: Author’s own work) 41
- 4.6 The final database structure used for the algorithm (Source: Author’s own work) 47

List of Tables

4.1	Datasheet structure: Organization	33
4.2	Datasheet structure: Target	34
4.3	Datasheet structure: Usage	34
5.1	Confusion matrix	49

Introduction

Background

Recommender systems are nowadays used more than ever. Their purpose is to recommend new music, movies, news articles, or even partners on dating apps. These systems are trying to understand tastes and desires of users to find the best content that suits them. But understanding human preferences is not a simple task to solve with an automatic program. Although there have been many successful implementations. The best known ones nowadays are Netflix, Amazon, and Instagram, but there are many others. These sites have found a way to sell more, entertain more, or make the user use their app more, by recommending content and products to their users.

Purpose

The purpose of this thesis is to design and program a functional automated recommender system that proposes potential customers to sales representatives using real data. The system has to be usable in real world use case. Therefore implemented in a scalable, re-usable and effective way.

Goals

This thesis proposes several goals that are to be met:

- Describe the phenomenon of recommender systems
 - Introduce the history of recommender systems
 - Describe the current solutions and implementation approaches
- Create an automated system for recommending potential customers based on user preferences or interactions
 - Design a local database to analyze and store input data
 - Design and run own functional algorithm
 - Create an interface for the developer to setup and run the algorithm

Added benefits of the thesis

This thesis should serve as a brief example on how do modern recommender systems work, how they are designed and how to create one.

Research

The main research for this thesis was conducted by analyzing a number of published books, academic works, and online articles on the topic. There were a number of bachelor's theses published on the topic of recommender systems in the Czech Republic, few of them even using machine learning mechanisms. Most of them were published in the past 3 years which indicates the rising popularity and therefore increasing amount of research on the matter of recommendation systems. What makes the topic of this thesis different is the practical focus on the B2B sales segment with the use of real-world data.

Practical Recommender Systems

A major initial source of information and know-how was the book *Practical Recommender Systems* by Kim Falk.⁽¹⁾ Kim Falk is a senior data scientist with years of experience with machine learning, natural language processing, or recommender systems. His book defines what recommender systems are and how are they designed. A large part of the book consists of practical examples in Python code.

Recommender Systems Handbook

An important source of the theoretical part of this thesis is the book *Recommender Systems Handbook* by Francesco Ricci and others.⁽²⁾ This publication offers a wide and thorough explanation of modern methods and techniques used to create recommender systems, as well as practical examples in Java programming language.

Document structure

The document starts by defining the term recommender system. Then continues to introduce it's history and development while describing some of the important historical and current implementations. After that, the process of designing a recommender system is described in detail, from initial approach, to decisions about the algorithm formula. All while emphasizing the practical usability of the system in real world use case of B2B sales. After that, the implementation itself is broadly described in detail. Lastly, the document finishes with a discussion about the resulting system and its possible improvements.

1. Evolution of recommender systems

The concept of using an algorithm to recommend content to users is still relatively young and will be developed much further in the future. This chapter takes a brief look at the history of recommender systems, how they changed, where were they used, and how the methods have evolved. But first of all, let's define what a recommender system is.

1.1 What is a recommender system

A recommender system, sometimes called a recommendation system or a recommender engine, has many different definitions and descriptions. In a very general context, we can say that it is an algorithm that provides relevant content to its users. It tries to predict the user's preferences based on their historical behavior and other information connected to their profiles. (2)

The original purpose is to help users find what they are looking for faster and easier. Nowadays these systems are powerful and complex tools used in many industries and contexts. From music and movies to news articles, online dating, or even financial services. With increasing volumes of data and the complexity of algorithms, technology like machine learning or artificial intelligence is now being used. (3)

To make understanding these systems a little easier, let's look back at how they started and how the methods got more complex over time.

1.2 History and development

1.2.1 Digital bookshelf

The origin of recommender systems lead to Swedish professor Jussi Karlgren and his tech report called An Algebra for Recommendations.(4) In this work he tries to measure the proximity of digital documents by using data from readers. Proximity is calculated from the interest of readers. The reader of document A is asked if he liked the document and if he thinks he would like other documents (B, C, ...). All the results are then saved into a matrix that represents the interest of one reader in a document and his interest in all the other documents based on the one he read. The next step is to sum the reader's ratings to one matrix for each document. The easiest way would be to assign each rating a numerical value and use regular arithmetic addition of all the matrixes.

Professor Karlgrens sent his original paper to the Interact magazine where it was rejected. The reason was that the system would interfere with the privacy of users. (5) Considering

the scale at which user data is used today, particularly in recommender systems, this might create a feeling of bitter irony.

1.2.2 Further development in the 1990s

Development of actual recommender systems came soon after Karlgrens tech report. Researchers Upendra Shardanand and Pattie Maes from MIT tried to use user data to recommend music.(6) They created an online service called Ringo, where users could rate albums and artists. Then used the information to recommend more music by using different algorithms, comparing both user-profiles and artist profiles.

Similar research was conducted by another MIT research team led by Paul Resnick, but with news articles instead of music. (7) This effort evolved into a research project called GroupLens which still exists in 2021. Many interesting recommender system applications were developed in the GroupLens research lab throughout the years, including movie and book recommenders, or analyzers of interfaces and tagging. (8)

1.2.3 The Netflix prize

Another important step in the evolution of recommender systems was an open competition held by the video streaming platform Netflix. (9) The goal was to create a collaborative filtering algorithm for predicting movie ratings. A training dataset of over 100 million ratings was provided and over 20 000 teams from over the world participated. The effort brought significant improvements and new insights into recommender algorithms. Additionally, the competition created a hub for knowledge sharing and cooperation, bringing together scientists and researchers from various fields and backgrounds. (10)

1.2.4 Current and future research

Researchers from the National Institute of Technology in Durgapur, India have tracked down current research trends in recommender systems in their publication *Recommender Systems: An Overview, Research Trends, and Future Directions*. (11) According to their analysis of over 1000 research papers, the research of recommender systems has peaked in 2013-2014. Since then, the number of publications is slightly decreasing, but using new technology might change that. According to the publication, the most researched types of recommender systems are those based on collaborative filtering (to be explained in the next chapter). The publication also suggests that future recommender systems might focus more on the daily lives of ordinary people. So far, recommender systems have been used mainly in business, but with cognitive computing and artificial intelligence coming to play, these systems might become an integral part of our lives. (11)

2. Architecture of recommender systems

This chapter describes how recommender systems are designed and implemented. Methods described in this chapter will later be used to design a simple recommender system.

2.1 Structuring a recommender system

There are many things to take into consideration when designing a new recommender system. One of the approaches to do exactly that was described in the book *Word of Mouse: The Marketing Power of Collaborative Filtering* by John Riel and Joseph A. Konstan(12), and further broken down and explained in the book *Practical Recommender Systems* by Kim Falk (1). The core of the approach is in analyzing 8 dimensions before creating the system itself. These dimensions are domain, purpose, context, personalization, opinions, privacy and trustworthiness, interfaces, and algorithms. Some of these dimensions are very close to each other, some even connected. Here is a quick overview to explain the meaning of each dimension:

2.1.1 Domain

The first dimension is the domain. Domain should describe the subject that is being recommended and the context surrounding it. The creator first asks themselves what level of responsibility will the system have and what will the cost of error be. The second question is on the scope - will the system recommend songs and movies or substantive issues like investment opportunities? The impact of a "bad" recommendation can therefore have a substantial impact on the lives of people. Analyzing the domain should help choose which interfaces to use, how strict the algorithms should be, and what significance does the outcome has in general.

2.1.2 Purpose

Every creator should also ask themselves what the actual purpose of the system is. And not only for the end-user, but for the creator/company as well. Is the system used to increase sales? Then what type of customer is the most profitable and how to target him? Trying to make the user use an app more? Then finding the most eye-catching and attractive content that will entertain the most users might be the answer. Finding the purpose should also determine how to measure the performance of the system, e.g. number of clicks, time that the user spends on the website, etc.

2.1.3 Context

The context dimension should help understand how and where the system will be used. When recommending health insurance, users will probably use the system once from their home, using their laptop or desktop computer, and they will closely study each recommendation. But when recommending nearby restaurants, then the user might be just on their smartphone in the middle of the street, trying to quickly find the most convenient place to eat. This helps determine how to adjust the system and the way the results are presented.

2.1.4 Personalization

This dimension determines how personalized the recommendations should be. There are three main types of personalization:

Non-personalized

The non-personalized system is a purely statistical one. It is the simplest type of recommendation system. Usually, it is just a list of the most viewed, most liked, etc. All users see the same content.

Personalized

A personalized system, on the other hand, focuses on each user and analyzes every user's history of interactions with the data. Then creates a specific list of recommendations that is different for each user. These systems are usually the most complicated ones.

Segment-personalized

Segment-personalized systems are somewhere in the middle. They do not have any personal data about specific users and their interactions but can divide them into specific groups or segments. These segments can be based on location, age, sex, etc., or on some domain-specific attribute, e.g. what courses does a student study in the current semester. This information is not acquired by analyzing the user behavior, but usually just by asking the user a few questions when first using the system. All the users in the same segment get the same recommendations.

We can often find more than just one method on a single website. For example, YouTube main page shows videos that are based on user's history (personalized recommendation), Trending page shows videos trending in users region or country (segment-personalized rec-

ommendation), and then every channel has Most Popular sorting that sorts videos based on the number of views (non-personalized recommendation).

2.1.5 Opinions

Experts might be needed to make proper recommendations in some industries or contexts. In that case, who is going to be the expert and how much weight will the expert's opinion have must be decided. This is not used very often, but in some very specialized industries, having an expert rating can have a positive impact on the quality of the results.

2.1.6 Privacy and trustworthiness

Privacy is a major concern when it comes to user data. First of all, the algorithm has to work with all user data in a controlled and safe environment, making sure no data leakage happens. Second, the user has to be aware of the fact, that his data is processed by the app or website and should understand the purpose.

The user should be able to see the trade-off, for example, if Netflix analyzes user's data, they will get recommended better movies. If the user is not aware of his data being collected and analyzed or does not see the purpose, that's when the system is at risk of losing trustworthiness. For example, a user talks to a friend on a chatting app about his back hurting and the next day ads about pain pills start showing on their social media. At that moment the user might completely lose trust in the chatting app and switch to a different one.

2.1.7 Interface

Interface dimension is about the input and output of the system.

Input

Systems can collect data in two ways - explicit and implicit. Explicit data collection means asking the users for their opinions, ratings, etc. An example of that would be YouTube likes and subscriptions. Implicit data collection is done by analyzing user behavior and interactions.

Output

The output dimension describes how the results of the system are presented to users. This should go hand in hand with the Purpose and Context dimensions. E-shop presents recommendations beneath the shopping cart page based on the items already in it, the app for

restaurants shows nearby bistros right on the home page, and some apps even change the way they work based on previous interactions.

2.1.8 Algorithm

The last section is about the implementation of the system. There are various ways to write the algorithm, but the main three are collaborative, content-based, and hybrid.

Content-based system

A content-based system uses data about the items that are being recommended. This method is usually used when there is data about the content, but not about the users. The system starts by looking at content that the user liked or interacted with and creates a user profile. Then it tries to find similarities between content and compares the results to the user profile, filtering only the best matches.

Methods like TF-IDF and LDA models are used for rating the items based on their attributes. Similarities between documents can be calculated using cosine similarity, Jaccard distance, LP-norms, or Pearson's correlation coefficient. (1)

There are several limitations to the content-based systems. If items don't have enough interesting attributes, the system can't properly distinguish and filter them. Another problem can be the limited degree of novelty that content-based filtering offers. Because the system is based on user's preferences and ratings, it will always recommend something similar and never anything new or surprising. This problem is called *over-specialization* and can be problematic in some implementations, e.g. in movie and song recommendation.

Collaborative system

Collaborative systems are the most used and researched technique to build recommenders. (11) It calculates predictions by looking at filtering similar users or items. There are two basic ways to create a collaborative filtering system:

User-based filtering User-based collaborative filtering takes information about the user and finds users with similar parameters. The algorithm then predicts the user's rating by computing the weighted average of ratings provided by these similar users.

Item-based filtering With item-based collaborative filtering, the algorithm finds similarities between items first. Then it predicts the user's rating for an item by processing the user's previous ratings of similar items.

Both methods calculate the prediction based on values that are similar - these values are called the *neighborhood*. With user-based filtering, the rating is predicted using the predictions of neighboring users (users with similar behavior, demographics, etc.). Whereas in the case of item-based filtering, the rating is calculated by looking at the user's previous rating of neighboring items (e.g. movies in the same genre). There are several ways to determine the size of the neighborhood, those will be further explained in the next chapter.

Collaborative filtering systems can be problematic when it comes to new users or items. When there are no close neighbors, the algorithm might not be able to predict a rating well. This problem is also referred to as the *cold start problem*.

To better highlight the difference between collaborative and content-based systems, let's assume a system for recommending documents. Collaborative system would find similar users and recommend them documents that the other person has read. Whereas content-based system would analyse the documents and find similar ones to recommend. This example is visualized in the following schema.

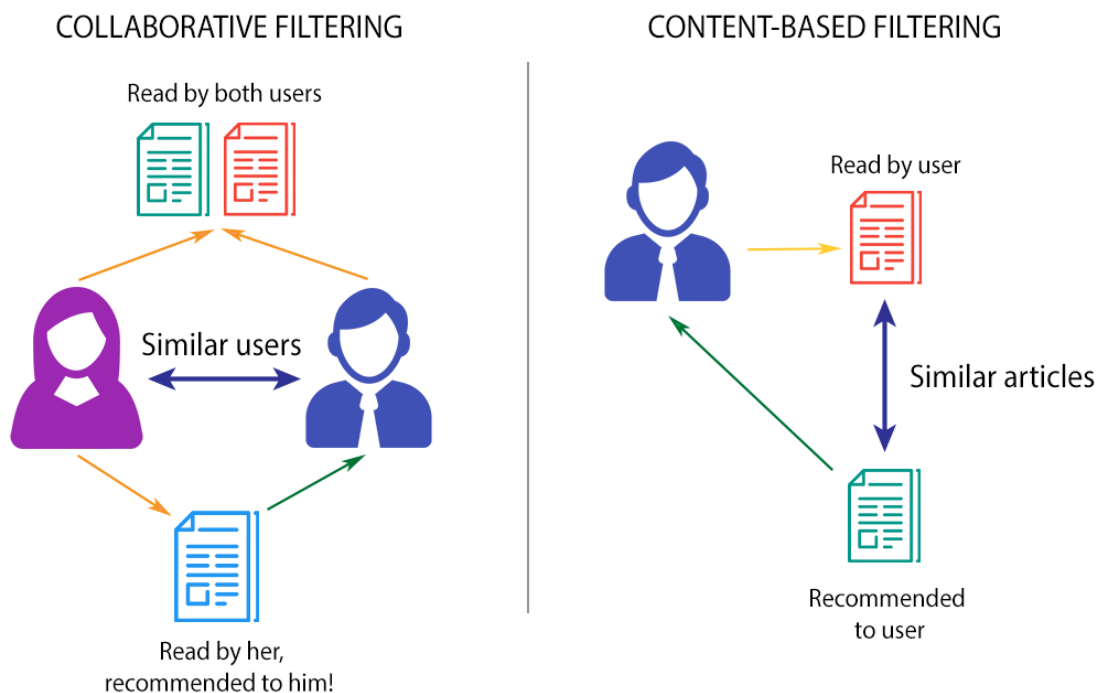


Figure 2.1: Comparison of collaborative and content-based systems (13)

Hybrid system

Hybrid systems are combining two or more recommending techniques together. Either by letting other recommender systems calculate results separately and then combining them or by implementing a system that uses both content-based and collaborative-based techniques. Combining both types of systems might be beneficial, because it can help to avoid some of the weaknesses of these separate systems, e.g. cold start problem, or over-specialization.

2.2 Chapter summary

In conclusion, there are several ways to approach the designing process of a recommender system. Each approach has its pros and cons and it depends on the specific implementation which approach should be chosen. Furthermore, these approaches can (and often are) combined to reimburse weaknesses.

The selection of approach for the implementation part of this thesis is explained and justified in the next chapter.

3. Designing a recommender system

This chapter describes the process of designing a recommender system for B2B data. First, it introduces the problem and the context around it, then describes the plan of implementation. The process of implementation itself is described in the following chapter.

3.1 Taxonomy of the B2B recommender

This project is built around a real company dataset about B2B sales. The company runs a website that could be described as a company catalog, with detailed information about companies aggregated from multiple data sources. Users (usually sales representatives) pay to access this catalog and gain additional information about companies they want to sell to or find new companies that might be their new potential customers. The dataset contains information about both the companies and the sales representatives.

Let's start by applying the approach of the 8 dimensions described in the last chapter.

3.1.1 Domain

The domain of this recommender system is companies and their data. The system analyzes details about the companies and then recommends them to particular users (sales representatives). However, this system is not the main source of information for the users. The significance is therefore not high, it is an experimental algorithm that could improve user experience, but it is not something the users would depend on.

3.1.2 Purpose

The purpose of the system for users is to help them find new leads. The purpose for the company is to improve its product for users to use it more.

Measuring success and impact

The specific measurement of the algorithm's success is a complex issue on its own. In the most general sense, the algorithm is successful when it increases user sales. That means recommending a company where the user then sells a product. But that information is not available to the system since it is not connected to every user's CRM. It is not possible to even know if the users contacted the company or how much they actually used the information provided on the page. Another option can then be looking at the user interaction with the company. The most simple interaction is clicking on the recommendation. That is a good

sign since it means the user thinks it might interest them. Monitoring user behavior on the company page might be another clue if the recommendation was good. The longer the users stay and the more information they collect from the page, the better. The actual analytics of how useful and impactful the algorithm is can be measured after its deployment into the real world. It is not something that can be precisely measured during the system's development.

3.1.3 Context

The website catalog is usually used by sales representatives from various industries. Since the user interface for this project will be only a simple testing website, no more information is needed. Implementing the system into the actual website catalog would have to be done separately and with more prudence.

3.1.4 Personalization

Since the users come from a diverse set of industries and companies, the recommendations should be personalized or at least segment-personalized.

3.1.5 Opinions

This dimension is not relevant in the context of this project. No experts for specific segments are needed, the algorithm will work with basic company data that has a consistent structure.

3.1.6 Privacy and trustworthiness

All the data used for this project is anonymized. Furthermore, the trade-off for users here is obvious - better company recommendations, therefore better deals. Data safety is a problem exceeding the scope of this thesis that would have to be solved while implementing the system into the actual website.

3.1.7 Interfaces

The main input was provided by a dataset, which is described in the next chapter. However, for success measuring, the system has to monitor if and how the user interacts with the recommendation. That information should then be added to the database and further analyzed. The output of the system will be a simple website page where the user gets a list of recommendations. In conclusion, the final website has to display data and also track user behaviour, to fulfill the purpose of the interface. The following section explains the architecture of the website, whereas the actual implementation is described in the next chapter.

3.2 Interface architecture

The interface is a simple website running on a local server that shows the user the company catalog, company detail, tracks interactions with companies, and shows the recommended companies. The purpose is to simulate the user behaviour and collect data about it. The secondary purpose of this website is to make development of the algorithm more convenient and create a site where the engine can be configured.

3.2.1 Functional requirements

To better specify the purpose of the interface some crucial functional requirements are defined:

- Website allows the developer to log in as any real user.
- Website allows scrolling through a catalog of companies.
- Clicking on a company to see its detail creates a new database row of a user-company interaction.
- The recommender engine can be run from the website.
- The recommender engine can be configured from the website.

3.2.2 Technology

The client-side is a simple HTML. Since the main recommender algorithm is coded using Python, the framework chosen to create the server-side is Django, a Python-based web framework. Django can be used to develop high-level web applications as well as small websites, it's easy to learn, versatile, and has a wide scope of features, therefore no additional external libraries are needed for the recommender website. With no external libraries, the code is more likely to stay clean, consistent, and there are no compatibility problems. Another benefit is that Django is free and open source.

3.2.3 Coding principles and design

Several principles were respected while developing the website for input and output. The Django framework uses a component-based architecture where each building block of the app is independent of the others. (14) Advantage of this approach is that each part of the website can be easily removed, replaced, or enhanced, making the code scalable in an efficient manner. Writing similar parts of the code independently does not mean duplicating code though, the universal DRY principle (Don't Repeat Yourself) should be respected as well. That can be achieved by grouping related functionality into reusable modules. All while respecting the SoC (Separation of Concerns) principle, which leads the programmer to keep distinct functionalities in separate methods, classes, files or even projects. While respecting these basic principles, the code should remain clean, understandable and scalable.

Django is based on the Model-View-Controller design pattern, which separates the code logic into three modules.(15) Model is used to define the data structure (e.g. database), View takes care of displaying the data (e.g. HTML, CSS, JavaScript), and Controller is a bridge between Model and View that handles their communication. The pattern used in Django is called Model-Template-View and it is very similar to the Model-View-Controller. The difference is that Django handles the Controller part itself and there is no need to implement it. The Template is a file that defines a layout and placeholders for data (e.g. HTML). The View then interacts with both Model, to get data, and Template, to populate the file with the data from the Model. The flow of data through the architecture can be seen in the following diagram.

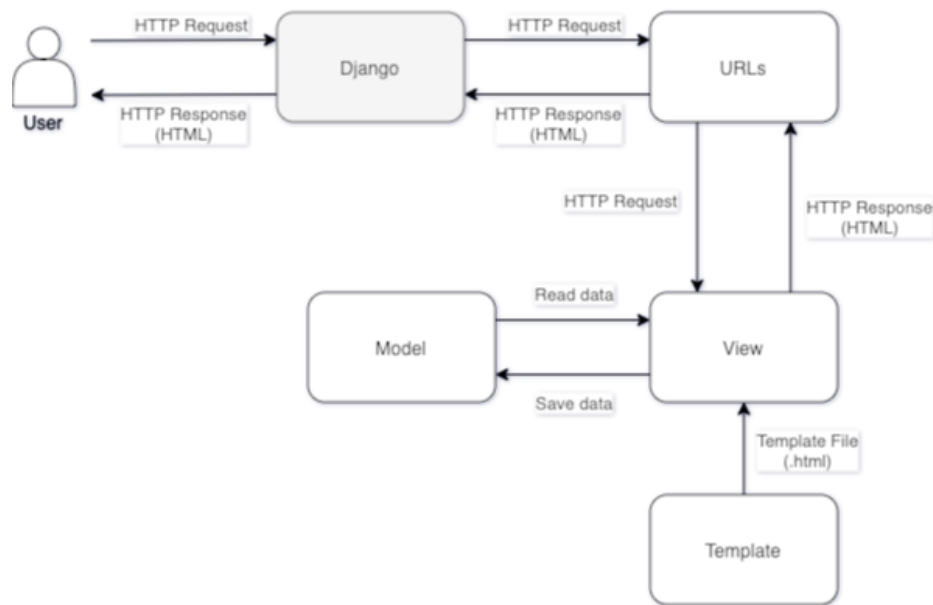


Figure 3.1: Data flow through the MTV Django architecture (Source: Author's own diagram)

3.3 Code style

In order to keep the code clean and consistent, some standard coding conventions should be respected. Since most of the code is written in Python, all these conventions are focused on it. This section lists the most important ones (naming, code layout, commenting) without much commentary. If an explanation of any of the rules is required, it can be found in the PEP 8 Style Guide for Python Code. (16)

3.3.1 Naming conventions

Naming should strike a good balance between being too descriptive and being too general. Package, module, method, function, and instance variable names are all lowercase with words separated by an underscore. Classes are named with uppercase camel convention (except for

Python built-in classes, those are usually lowercase). Constants are always all uppercase and words are separated with an underscore.

3.3.2 Code layout

Each indentation level is separated by 4 spaces. Use global maximum line length. Keep imports together on separate lines at the top of each file. Blank lines should be used to separate methods, functions, classes and one should be at the end of each file. Otherwise, white characters should be kept to a necessary minimum.

3.3.3 Commenting conventions

The code should be primarily understandable by itself, but if that is not possible, then comments can be used. In this code, only block and inline comments are going to be used. The comments have to always be written right above (or next to) the code that they are explaining.

3.4 Algorithm

The purpose of this algorithm is to analyze the input data and recommend companies to users. Or in other words, try to predict in what companies would the user be most interested in. The first step is choosing the filtering approach.

Collaborative filtering could be used, but there are several issues that would come up. The main problem is that there is not much detailed data about users provided, therefore it would be problematic to find similarities between them and use it as a basis for recommending to each other. The cold start problem would occur as well since new users will not have any interactions with companies. Even though there are possible solutions that solve the cold start problem to various degrees, most of them would require extra development. For example, creating a new web page, where a new users would select from a list of random companies those, in which they are interested, to get some initial interactions.

For these reasons, the content-based approach was chosen as the most convenient solution. It is based mainly on attributes of items (companies) and therefore is less prone to the cold start problem, and the lack of user data is not a significant problem either. Furthermore, the content-based approach is more simple to implement and start with, the collaborative approach can be later added to create a hybrid system and further improve the filtering algorithm.

3.4.1 Elements of the algorithm

As described in the second chapter of this thesis, content-based filtering is based on item features and the user's interest in them (interactions). In the case of this project, items are represented by companies. Even though there is not much data about the user, simply having the user accounts is enough to create user profiles with individual preferences. Therefore, the main two elements the algorithm will operate with will be companies and users. Both can be represented by a vector.

3.4.2 Vector representation

Representing items and users with vectors is convenient because it opens lots of possible solutions for how to calculate predictions and recommendations. Each dimension of a vector represents one company attribute. It is important that all the companies have the same amount of attributes, otherwise operations like scalar product would not work (scalar product is explained in the following section). If some information about a company is missing, that fact should be expressed by the value of the attribute, not by the lack of it.

3.4.3 Vector distance

The vectors can then be used to calculate their distance with dot product, vector normalization, and cosine similarity. The calculated distance of a user vector and a company vector can be a score (or a part of it) that is used to decide which companies will be recommended.

Dot product (scalar product)

Dot product can be defined either algebraically or geometrically. Algebraic definition is sufficient for the purpose of the calculations in this project. The result of this operation is a scalar number, which is why the dot product is also often called a scalar product.

Assume two vectors, where n is the number of dimensions:

$$x = (x_1, x_2, \dots, x_n)$$

$$y = (y_1, y_2, \dots, y_n)$$

Dot product for these vectors is then defined as a summary of multiplications of their dimensions:

$$x \cdot y = \sum_{i=1}^n x_i y_i = x_1 y_1 + x_2 y_2 + \dots + x_n y_n$$

Euclidean vector norm

A mathematical norm is for nonnegative real numbers that behaves can be used to represent a distance from an origin. For a vector $x = (x_1, x_2, \dots, x_n)$ with n dimensions on an Euclidean space (also sometimes called L^2 norm), the norm is defined as following:

$$||x||_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

Cosine similarity

Cosine similarity is used to measure the similarity of two vectors in a inner product space. Mathematical shortcut for cosine similarity is $\cos(\theta)$ and the function definition for two vectors goes as following.

Assume two vectors, where n is the number of dimensions:

$$x = (x_1, x_2, \dots, x_n)$$

$$y = (y_1, y_2, \dots, y_n)$$

The cosine similarity for these two vectors is equal to the division of their dot product and the multiplication of their norms:

$$\cos(\theta) = \frac{x \cdot y}{||x|| ||y||}$$

The result is a scalar number in the range of $< -1; 1 >$ where -1 means that the vectors are in the complete opposite direction, 0 means the vectors hold a 90° angle, and 1 means the vectors are in the same exact direction.

The cosine similarity is a way to measure the similarity of two vectors and therefore a way to measure the similarity of companies and user profiles that will be used in the algorithm.

3.4.4 User profile

A user profile is a vector that contains information about the preferences of a single user. It has the same exact structure as company vectors, so operations on the two can be done without constantly changing the structure.

Creating a user profile

There are countless ways to create a user profile. It all depends on how much data about the users and their interactions are available. If the user has a history of interactions (e.g. likes, ratings, etc) then iterating through that history is a good start.

The input dataset offers data about the user interacting with companies, which is a good start to create the user profile vector. All the interactions of a user are selected and the attributes of the company interacted with are saved. Then, average value of each attribute is calculated. This result becomes the new user profile value of that specific user profile. This is done for numeric attributes, like company size.

With attributes like NACE codes or legal form, the average value might not be the best solution. For example, a user might have interacted with companies that have two NACE codes: C - Manufacturing and G - Wholesale and retail trade. If the algebraic average was used to recommend other companies to the user, these two NACE codes would result in NACE code E - Water supply and sewerage, which is not remotely close to any of the two previous industries.

Therefore, instead of average value, both NACE codes and legal forms are recommended by using the mode function. In short, the mode function finds the most frequent value in a set. This solution has its weaknesses since it only works with the most used value and not with any others. However, it will be sufficient as a base ground for user profile creation that can be improved in the future.

3.4.5 Comparing data with tree structures

Since NACE codes and legal form codes do not have a numerical value, they can't be easily compared. However, they do have a tree structure and can therefore be handled as ordinal values. There are several options on how the codes could be compared.

Equality check

The most simple way is to check if the codes are equal. If so, both compared vector attributes are changed to the same value (e.g. 1). If not, the attributes are changed to opposite values (-1 and 1). This is simple, but not very effective. Some codes can be very close to each other, but not quite the same, and they will produce the same result as two codes from a completely different side of the tree. This distinction should be and is addressed.

To get a slightly better result, the comparison can be done in a higher level of the tree. Therefore, the codes that differ on the lower level, will result in a match. But then the information about the low level difference is lost. This method was used as a starting point, but it was not precise enough to yield satisfying results.

Tree structure

A way to get more precise results is to implement a function or a method that will find the distance between the exact nodes of a tree. The distance is in this case defined as a minimal number of edges needed to reach one node from the other one.

Calculating the distance between two nodes can be done with the following set of steps:

1. Find the path from the first node to root of the tree.
2. Find the path from the second node to root of the tree.
3. Compare the two paths to find the first common ancestor. The rest of the path after the common ancestor is the same for both.
4. Count and sum the number of edges in both paths before the common ancestor.
5. The result sum is the distance between both nodes in the tree.

To make the steps more clear, they can be simplified and put into the following formula.

$$Distance(x, y) = Distance(root, x) + Distance(root, y) - 2 * Distance(root, lca)$$

Where:

Distance(x, y): distance between the nodes x and y

x, y: nodes of the tree

root: the single root of the tree

lca: lowest common ancestor

The distances of both NACE codes and legal form codes will be calculated this way.

3.4.6 Randomizer

As mentioned earlier, one of the problems of the content-based approach can be overspecialization. If the algorithm is delivering only very similar items, it will never recommend something new or surprising. One of the ways to solve this issue is to bring a random element into the algorithm.

Let's say the algorithm has finished scoring all the companies, the list of results is sorted, and the top companies are selected. After all that, a few randomly chosen companies that are outside of the top selection will be added to the results. The range from which these random companies can be selected will be defined. This way, there won't be any completely random picks from a whole another spectrum of companies.

For example, if the range is 100, then the extra companies will be selected from the next 100 best scored companies after the top selection. When there are more companies than just the dataset the range could be defined as a percentage instead of a quantity.

3.5 Chapter summary

The main theoretical basis behind the algorithm has been introduced in this chapter. The content-based approach has been chosen as the main design approach and the most important parts of the calculation process have been drafted. The technology used to implement these ideas is mainly Python and Django.

4. Implementation

This chapter goes through the process of implementing a simple recommender system for provided B2B data.

4.1 Database and dataset

The domain of this project is data and there are several decisions to be made about how it is going to be structured and handled. The provided dataset might not have a very convenient structure for a Python program, therefore there are several changes to be done.

4.1.1 Initial dataset

The provided file contains data about companies operating in the Czech Republic. These companies are of various sizes, legal forms and markets. Then there is anonymized list of users and their historical interactions with the companies. The initial dataset was provided in a .xlsx file and contained three sheets - Organization, Target, Usage. It can be found in *Attachment A*.

Organization

The first sheet is called *Organization* and represents a company that uses the company catalog (are the clients of the company catalog). Data about the company provided is:

Table 4.1: Datasheet structure: Organization

Column name	Description
OrgCode	unique hash
NaceCode	code number according to the European classification system of economic activities (NACE = Nomenclature des Activites Economiques)
LegalFormCode	numerical code according to the Czech standard of legal forms
Size	numerical parameter that was calculated from the company turnover, number of employees, and capital value. The formula for calculating this parameter is not public

Target

The second sheet is called *Target* and represents the firms in the company catalog, therefore the companies that are supposed to be recommended to the users belonging in the Organization table. It consists of the following columns:

Table 4.2: Datasheet structure: Target

Column name	Description
TargetId	unique numerical identification code
Name	the registered name of the company
NaceCode, Legal-FormCode, Size	last three columns are the same as in the Organization table

Usage

The last sheet represents interactions between users and companies. Every time a user clicks on a company, a new row is created in this table. These columns are provided:

Table 4.3: Datasheet structure: Usage

Column name	Description
ID	unique numerical identification code
SourceId	a numerical 4-digit code of a person from the Organization. Is unique for a single user of the company catalog belonging to one of the client organizations
TargetId	the code of the company that the user clicked on
CreatedAt	date of the interaction
OrgCode	the code of the organization that the user belongs to

There are 121 rows in the *Organization* sheet, over 2500 rows in the *Target* sheet, and over 62000 rows in the *Usage* sheet. If the dataset was transformed into a conceptual database the schema would look like this:

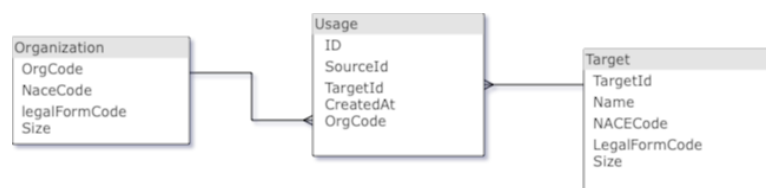


Figure 4.1: Conceptual structure of the provided dataset (Source: Author's own work)

4.1.2 Database design

Since most of the development is happening in Python, a database is a better way to work with data than with an Excel file. Therefore, a local database server has been created using SQLite database engine (version 2.6.0). The first reason to choose the SQLite library is the fact, that it is open-source and free to use in a commercial environment. Furthermore, it is one of the most popular database systems, thus having a large community, thorough documentation, and is deployable to all major operating systems. (17)

4.1.3 Structure transformation

There are several problems with the dataset structure. First, the NACE and legal form codes have no names, and understanding their value can be beneficial during the data analysis. For that reason, two new tables will be added, one for each code, that will contain their descriptions. Another problem is that user administration will be needed and getting users from the *Usage* table is not very effective. A separate table for users is created for that reason. The last issue is the naming of tables and columns which is not consistent. Furthermore, some names could be improved so the purpose of the tables is more clear. For example, the *Target* table becomes *Company* and the *Organization* is renamed to *ClientOrganization*. Several columns will be renamed in a similar fashion. The final version of the database structure can be seen in picture 4.2, where private keys are indicated by bold font and foreign keys by italic.

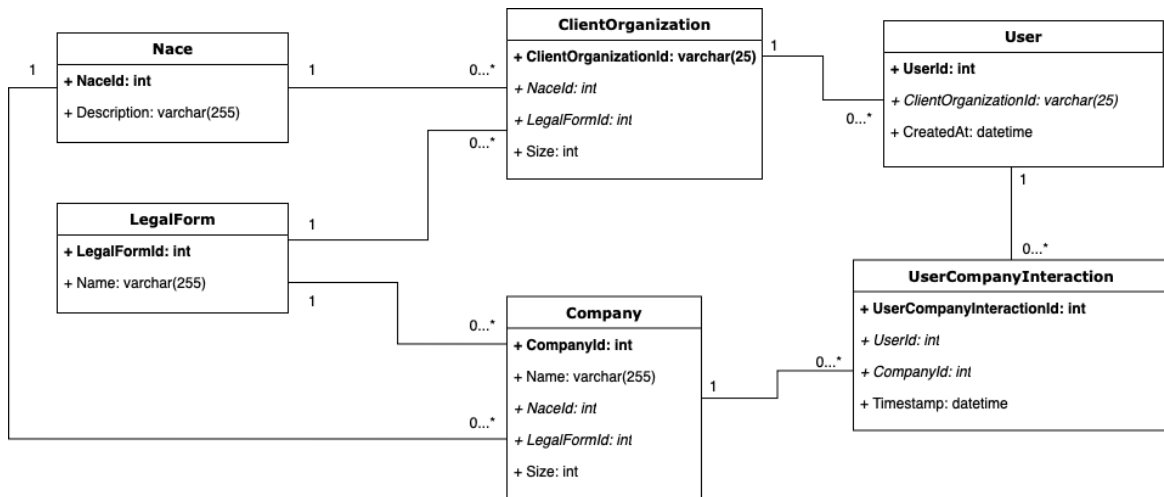


Figure 4.2: Physical diagram of the final database (Source: Author's own work)

The database is created and administered with a code-first approach. That means the database entities are first created and configured as Python classes, the actual database entities are later created using automated migrations. More on this topic in the next section about code structure in the subsection Migrations and Models.

4.1.4 Loading data to the database

After the database is created, it has to be filled with data from the Excel file. That was achieved by exporting the separate entities and their data to CSV (Comma Separated Values) files and then writing a short Python script that loaded them into the actual database. The conversion to CSV was done because Python comes with an easy-to-use library that allows the processing of CSV files in a more convenient way than with other Excel formats. (18) The conversion itself is very simple and fast to do in Excel, causing almost no time delay.

Using the Python library for handling CSV files, the code for inserting separate entities was almost similar. The only exception was the number and names of specific attributes. Here is an example of how the code looks like for the entity *ClientOrganization*.

```
import csv, sqlite3

con = sqlite3.connect('db.sqlite3')
cur = con.cursor()

with open('client_organization.csv','r', encoding="utf-8-sig") as fin:
    dr = csv.DictReader(fin, delimiter=';') # comma is default delimiter
    to_db = [(i['ClientOrganizationId'], i['NaceId'],
                i['LegalFormId'], i['Size']) for i in dr]

cur.executemany("INSERT INTO catalog_clientorganization (ClientOrganizationId, "
                "NaceId, LegalFormId, Size) VALUES (?, ?, ?, ?);", to_db)
con.commit()
con.close()
```

This is a solution created solely for the purpose of uploading the initial data to the database. In a real use case, the data would be inserted through different channels. For example, users are created with another app used by client support, and companies are created and updated by stored procedures that process analysis of public company data published by local authorities.

4.2 Interface code structure

4.2.1 Website Implementation

The implementation of the architecture is explained in depth in this section. The website consists of three main parts: *base_site*, *catalog*, and *settings*. The structure of the actual code can be seen in the following picture.

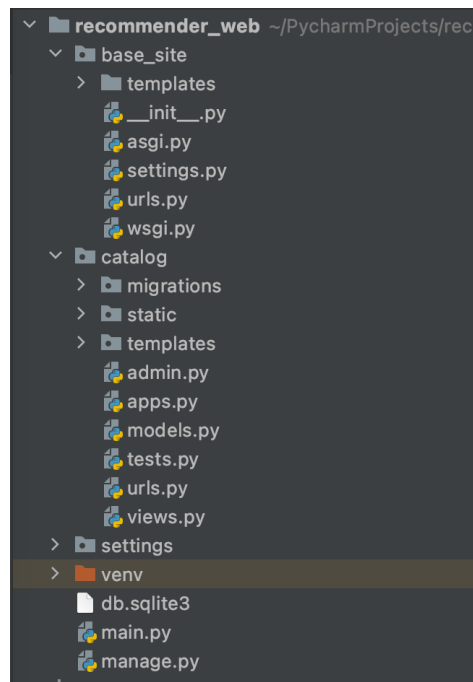


Figure 4.3: Code structure of the interface implementation (Source: Author's own work)

Base site

The *base_site* directory represents the main building block of the website. Any code that will be used for the whole website should be written here to respect the DRY concept. That means static parts of the design, like the navigation menu, or aggregating and routing all the children's directories.

Settings The *settings.py* file contains all the necessary configurations of the Django project. Most importantly, it defines a list of projects and directories used in the project, in this case the *catalog* and *settings* directories. Other configuration includes setting the project global language and timezone, choosing middle-ware libraries (e.g. for authentication, sessions, etc.), or defining the default URL address.

URLs The *urls.py* file is very simple, but important nonetheless. Its purpose is to register URL patterns and connect them with views. Different versions of this file are used in all three project directories.

Templates There is only one template called *base.html*, that consists of the navigation menu at the top of the website. This menu is the same everywhere on the website. The second important part of this file is importing Bootstrap, which is the only external library this website uses. Bootstrap is a popular front-end library with pre-made CSS classes, that will make styling the website way faster and easier. Even though design is not the main focus in

this project, since the website is not used by actual users, but only by a developer, it is still useful to have at least a clear website layout.

Catalog

The *catalog* directory defines the views, models, and templates for the part of the web where companies are shown. That means two main views - catalog index and catalog detail. The first describes the company catalog, a page where a list of companies is shown to the user. The second one is a page of a single company, that is shown after the user clicks on one of the companies in the catalog.

Here is an example of a template file *catalog_detail.html*:

```
{% extends "base.html" %}
{% load static %}

{% block page_content %}
<h1>{{ company.Name }}</h1>
<div class="row">
  <div class="col-md-1">
  </div>
  <div class="col-md-4">
    <h5>Legal form:</h5>
    <p>{{ company.LegalForm }}</p>
    <br>
    <h5>NACE code:</h5>
    <p>{{ company.Nace }}</p>
    <br>
    <h5>Size:</h5>
    <p>{{ company.Size }}</p>
  </div>
</div>
{% endblock %}
```

The code starts by importing the file *base.html*, which includes the website elements common for all the templates (top navigation menu, importing Bootstrap, etc.). The second row loads elements, that are always shown in this template (e.g. images, or static data). All the HTML code of this page is inside a *page content* block. That makes sure that this template is connected to the right place of the base template. The variable data from Python context are accessed by using the double curly brackets.

The final catalog page can be seen in a following screenshot. The navigation menu at the top of the page comes from the base site file.

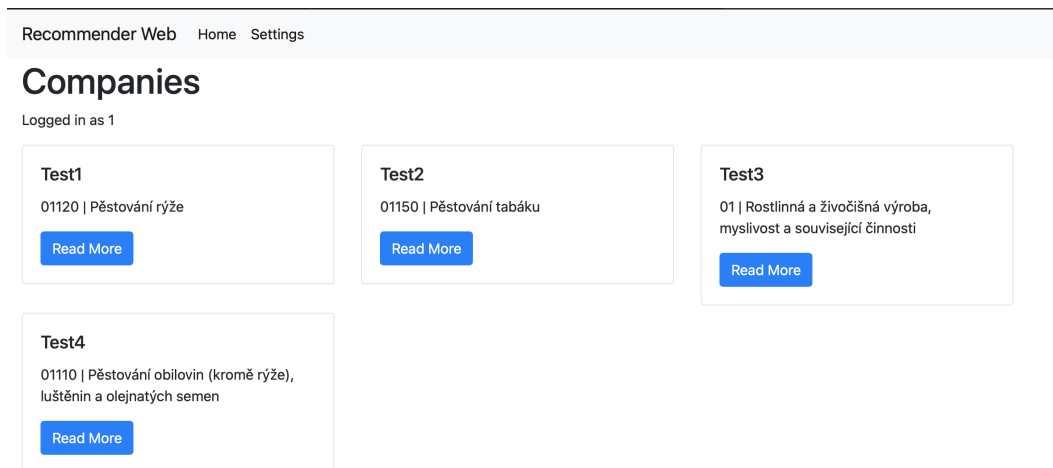


Figure 4.4: Catalog index website with test values (Source: Author's own work)

Pagination Since there are hundreds of companies in the database just from the input dataset listing them all in one page is not a valid option. It would lower the clarity of the page, user could be overwhelmed, and it could potentially lead to performance issues. One solution is splitting the data into separate pages that the user can list through. Django offers a class called *Paginator* that gets most of the job done by distributing the input data into separate *Page* objects.

The *Paginator* class is imported and used in the view definition, in this case in the *catalog_index* view. The usage consists of initiating the *Paginator* class and setting the input data and a page length. Lastly, a try-catch block is used to prevent problems with the request form while setting the specific page number. The usage looks as following:

```
from django.core.paginator import Paginator, PageNotAnInteger, EmptyPage
...

def catalog_index(request):
    ...
    companies = Company.objects.all()
    page = request.GET.get('page', 1)

    paginator = Paginator(companies, 15)
    try:
        companies = paginator.page(page)
    except PageNotAnInteger:
        companies = paginator.page(1)
    except EmptyPage:
        companies = paginator.page(paginator.num_pages)
```

The *Paginator* is now functional, although the control elements are still missing. Those have to be defined in the HTML template. Attributes like *page_range*, *next_page_number*, and

others can now be accessed in the context of the *companies* dictionary. By looping through the range of data, a calculated number of page elements is created. Those are then used as control elements to list through the pages. This is what the code in the *catalog_index.html* file looks like:

```
{% if companies.has_other_pages %}
<ul class="pagination">
  {% if companies.has_previous %}
    <li><a href="?page={{ companies.previous_page_number }}">&laquo;</a></li>
  {% else %}
    <li class="disabled"><span>&laquo;</span></li>
  {% endif %}
  {% for i in companies.paginator.page_range %}
    {% if companies.number == i %}
      <li class="active"><span>{{ i }} <span class="sr-only">(current)
      </span></span></li>
    {% else %}
      <li><a href="?page={{ i }}">{{ i }}</a></li>
    {% endif %}
  {% endfor %}
  {% if companies.has_next %}
    <li><a href="?page={{ companies.next_page_number }}">&raquo;</a></li>
  {% else %}
    <li class="disabled"><span>&raquo;</span></li>
  {% endif %}
</ul>
{% endif %}
```

Migrations and Models This directory is the place where models are defined. The database design from the previous chapter is compliant with these models, which were created first. The database is created by generating migrations. Migrations are an automatic way of applying the models into the database. By running Python commands *migrate* and *makemigrations* a file with all the changes is created. That file is then transformed into the language of the database system used for the project and executed.

An example of a model *Company* class can be seen here:

```
from django.db import models

class Company(models.Model):
    CompanyId = models.AutoField(primary_key=True)
    Name = models.CharField(max_length=255)
    NaceId = models.ForeignKey(Nace, on_delete=models.CASCADE)
    LegalFormId = models.ForeignKey(LegalForm, on_delete=models.CASCADE)
```



```

Size = models.IntegerField()

def __str__(self):
    return self.name

```

Each parameter of the model represents a database field. It is defined by using the *django.db.Models.Model* library and its subclasses. For example, the Name parameter is defined by the *CharField* subclass, and foreign keys are defined by the *ForeignKey* subclass. Different subclasses can have different input variables. For example, for defining maximum length of a *CharField* or for defining the origin of a foreign key. At the end of the example code, the method for representing class objects is overridden to return the name of the company. This is done for the purpose of more developer friendly debugging, Python would otherwise return only object id number. Other models return different attributes.

Settings

The last directory is called *settings* and it defines the administrator part of the website. The purpose of this part of the web is purely to make development easier. The developer can choose to log in as a different user here, run the recommender algorithm with various configurations, and see its output log.

Recommender Web Home Settings

Engine configuration

Select active user:

Save active user

Maximum company size:

Neighborhood size:

Randomizer size:

Randomizer range:

Calculate

Last run log:

Company	User	User Vector	Score	Is Random	Timestamp
Test3	1	7 01 112	1.0	False	May 5, 2021, 9:26 p.m.
Test4	1	7 01 112	0.4421837668553588	False	May 5, 2021, 9:26 p.m.
Test1	1	7 01 112	0.3273877136628122	True	May 5, 2021, 9:26 p.m.

Figure 4.5: Settings page with engine configuration and last run log (Source: Author's own work)

The structure of the Settings project is very similar to the catalog. The only interesting difference is the use of *forms.py* file. This file is used to define forms that users can input data with. In this case, the forms are used to get data about the algorithm configurations.

The screenshot above shows what the settings page can look like. The administrator can choose to log in as a specific user with the first form. Then the recommender engine can be run with various settings. The last part of the page is left for a table where the administrator can see the results of the last run.

4.2.2 Algorithm variables

The recommender engine has several variables that can be changed in the website interface. These variables (and data) are inputs of the recommender engine and can change the results in a significant way. There are 4 variables that the administrator can change - maximum company size, neighborhood size, randomizer size, and randomizer range. These variables can be set on the *Settings* page of the website interface.

The maximum company size attribute can be set to determine the maximum value that a company size can be. Since the maximum value is not provided in the initial dataset, this value is used as a reference point to determine if the company is large, medium, or small by using the transformation formula. The default value of this attribute is *1000*, which is the highest value of the size attribute in the whole dataset.

The attribute neighborhood size defines how many items are going to be recommended (outputted). The randomizer size determines how many recommended items are going to be added by the randomizer algorithm. And the randomizer range determines how far from the neighborhood can the randomizer reach.

4.3 Implementation of the algorithm

The algorithm is implemented in a separate library *recommender_engine* which is called when the administrator runs the algorithm from the Settings page. The algorithm is separated in order to respect the separation of concerns principle, making it reusable in a different framework. Since the user interface created for the purpose of this project is only for development purposes, the eventuality of using another framework to access the algorithm engine is highly probable.

4.3.1 Vector class

Most of the calculations are done using vectors. To assure that the vector structure is the same in all instances, a class that defines what variables are used is created in one place. Since the structure of both company and user profile vectors is the same, a single class can be defined for both of them. The class itself is fairly plain, it only consists of a set of attributes describing the company features used in the calculations. The file that contains the class also

includes some methods that help with processing the attribute values, for example, a method for normalizing values.

Value transformation

To keep intermediate results clear, some values of the vectors will be transformed, or normalized, so that all the attributes are in a range of $< -1; 1 >$. This concerns for example the attribute describing the size of companies. There is no maximum size defined and the largest number in the provided dataset can be differ in another dataset.

The formula to achieve this can be done using the mathematical operation of linear transformation on a random variable. Linear transformation is a mathematical function that transforms from one vector space to another while respecting the underlying structure. Applying it to a random variable, results in a new random variable. (19)

To create a formula that can be applied to attributes used in the database, the minimal value, the maximal value, and the value to be transformed are all needed. Transforming value x from a set of values X would then be calculated as following:

$$x = 2 \frac{x - \min(X)}{\max(X) - \min(X)} - 1$$

In the code, this formula is implemented in a single function. Instead of *min* and *max* set functions, the values are provided as input variables. The method looks as following:

```
# Normalizes value to range of <-1; 1>
def transform_value(value, min, max):
    return 2 * ((value - min) / (max - min)) - 1
```

4.3.2 Comparing NACE codes

As described in the previous chapter, comparing NACE codes is done by calculating their distance in a tree. The implementation of the algorithm consists of several steps.

First, the current structure of a NACE code is not ready to be a tree data structure, since it only contains private key and description. To solve this, a foreign key with id of a parent needs to become a part of the NACE database entity. That way, the tree structure can be constructed with NACE codes as its nodes.

With the structure ready, the algorithm can be implemented in the code. The first part of getting paths of compared nodes to root is realized with a short recursive method:

```

def get_path_to_root(self, node, path):
    path.append(node['Id'])
    parent = self.get_node_by_id(self.nace_tree, node['Parent_id'])
    if parent is None:
        return path
    return self.get_path_to_root(parent, path)

```

The recursive method takes two input parameters. The first one is the current node that keeps changing on each recursion level until it reaches root. The second parameter is a string array where each string contains id of a node that was on the path to root.

With this method to calculate the paths finished, the rest of the formula can now be implemented. The method starts by finding the node paths, then comparing them, finding their lowest common ancestor, and finally calculating the distance and returning its normalized form.

```

def get_nodes_distance(self, start_node, goal_node):
    # Get node paths to root
    start_path = self.get_path_to_root(start_node, [])
    goal_path = self.get_path_to_root(goal_node, [])

    # Find the first common parent
    for node in start_path:
        if node in goal_path:
            goal_length = goal_path.index(node)
            start_length = start_path.index(node)
            distance = goal_length + start_length
            return self.transform(distance, 0, self.max_nace_depth)

    return 0

```

4.3.3 Calculating user profile vectors

As described in the previous chapter, user profile vectors determine preferable values of attributes of a single user. To calculate this vector, a method was created that receives two inputs - user interactions, and companies that were interacted with (to avoid loading the whole company table).

```

def calculate_user_pref_vector(self, interactions, companies):
    sizes = []
    legal_forms = []
    nace_codes = []

```

```

for interaction in interactions:
    company = companies.get(Id=interaction.Company_id)
    sizes.append(company.Size)
    legal_forms.append(company.LegalForm.Id)
    nace_codes.append(company.Nace.Id)

avg_size = round(sum(sizes)/len(sizes))
mean_legal_form = mode(legal_forms)
mean_nace_code = mode(nace_codes)
preference_vector = Vector(avg_size, mean_nace_code, mean_legal_form)
return preference_vector

```

The iterates through provided user-company interactions and saves the attribute values into separate arrays. After, the vector is created out of average or mode values in the arrays.

4.3.4 Calculating cosine similarity

The cosine similarity value of two vectors, user profile vector and company vector, is the main score of the engine. The algorithm works in a following way:

```

# Calculates cosine similarity of two company vectors
def predict_cos_sim(self, user_vector, company_vector):
    # Step 1: Calculate distances (Nace and LegalForm)
    company_legal_form_dist =
        self.get_nodes_distance(self.get_node_by_id(self.legal_form_tree,
                                                    user_vector.legal_form),
                               self.get_node_by_id(self.legal_form_tree,
                                                    company_vector.legal_form))

    company_nace_dist =
        self.get_nodes_distance(self.get_node_by_id(self.nace_tree,
                                                    user_vector.nace),
                               self.get_node_by_id(self.nace_tree,
                                                    company_vector.nace))

    # Step 2: reconstruct user and company vectors with new values
    user = [0, user_vector.size, 0]
    company = [company_nace_dist, company_vector.size, company_legal_form_dist]

    # Step 3: Calculate score
    cos_sim = dot(user, company) / (norm(user) * norm(company))
    return cos_sim

```

The method receives user profile vector and company vector. Values of these two vectors are compared throughout this whole process. The first step is to calculate the distances of

NACE code nodes and legal form nodes. The vectors are then reconstructed so that the user vector has zeros for the tree values, whereas the company vector has results of the distance calculations for the same values. The score is then calculated using *dot* and *norm* functions from a library called NumPy. NumPy is a Python package with a vast selection of mathematical tools and several other tools and it is used in the code several other times. (20)

4.3.5 Randomizer

Implementing the randomizer into the algorithm has been done in a following way:

```
# Add randomized companies
x = 0
while x < self.randomizer_size:
    rand = random.randint(0, self.randomizer_range)
    rand_index = rand + self.neighborhood_size

    if rand_index > (len(results_sorted) - 1):
        rand_index = len(results_sorted) - 1

    if not results_sorted[rand_index].is_recommended:
        results_sorted[rand_index].is_recommended = True
        results_sorted[rand_index].is_random = True
        x = x + 1
    print("Randomizer adds company with id " +
          results_sorted[rand_index].company_id)
```

The *results_sorted* array contains objects with information about the results for each company. The array is sorted from the best score to the worst. A random index between the last recommended company and the randomizer range is generated. The company with this index is then set as recommended and also as randomly recommended to avoid confusion when debugging.

4.4 Chapter summary

The implementation consists of more steps, however, the most important ones were covered in this chapter. The main parts of the algorithm are described as well as some changes to the database structure. The full source code is stored in the **Attachment B**.

The results of the algorithm are saved to the database as well. An extra table called *User-CompanyRecommendation* has been created for that purpose. The table holds information

about the specific algorithm run, user, company, the specific user profile vector that was used, the calculated, and a timestamp of the run.

As mentioned earlier in the chapter, the NACE and legal form tables have been used as tree structures. For that reason, an attribute with their parent id had to be added as well.

The final database structure used can be seen in the following diagram:

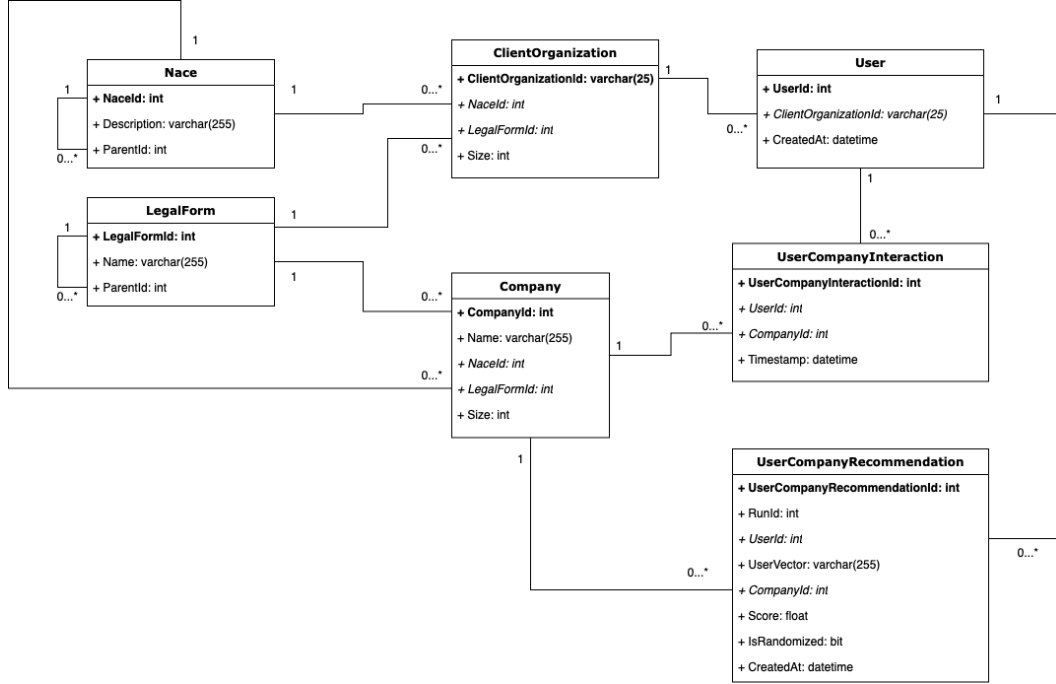


Figure 4.6: The final database structure used for the algorithm (Source: Author's own work)

4.4.1 Calculation showcase

Lastly, to better summarize the process and results of the algorithm, an example taken from the input data is showcased.

A user has 5 historical interactions with 3 unique companies. All 3 companies have different NACE codes, although all of them are in a tree branch *01: Crop and animal production*. Their legal forms are all *112: Limited company*, and their sizes vary between 5 to 10. These interactions result in a user vector [7, 01, 112].

This user vector is compared to all the other company vectors available and their cosine similarities are calculated. The best-found result is a company with a vector [12, 01, 112] and the final score of cosine similarity approximately 0.98. This company is obviously recommended as the best match, several other companies with slightly lower scores are recommended as well.

After that, a company is randomly selected within the randomizer range and added to the recommendations. This company has a size of 10, NACE code *0112: Growing of rice*, and

legal form *117: Partnership*. The cosine similarity of this company vector and the user vector is approximately 0.32 which is not the best but also is not completely off. The user is therefore recommended several very similar companies, as well as one that brings some level of novelty.

5. Evaluation

5.1 Algorithm

The algorithm is designed using a content-based approach which works well for the provided dataset. The real case is going to be more complicated though. There are going to be many more company attributes, probably more information about the users, and more information about the client organizations.

However, the goal was to create a solid basis to build upon. Since the code is written in a scalable and reusable way, that goal is met. The algorithm itself is functional and proposes potential customers. The question is, however, how good are the propositions? That question is in this case hard to answer. Let's first look at how are recommendations usually evaluated.

5.1.1 Recommender system quality metrics

Evaluating the results of a recommender system is a complicated problem. There are many metrics that can be used to measure the quality of recommendations, but the main two are precision and recall. (21)

Variables used to calculate both these metrics are better understood when visualized in a so called *confusion matrix*. This matrix is a standard way to visualize the performance of classification algorithms.

Precision Precision is sometimes also called a positive predictive value. The precision is defined as a fraction of relevant instances among the retrieved instances.

$$precision = \frac{TP}{TP + FP}$$

Recall Recall, also called sensitivity, is defined as a fraction of relevant instances that have been retrieved over the total amount of relevant instances.

		Actual	
		Positive	Negative
Predicted	Positive	<i>True Positive (TP)</i>	<i>False Positive (FP)</i>
	Negative	<i>False Negative (FN)</i>	<i>True Negative (TN)</i>

Table 5.1: Confusion matrix

$$recall = \frac{TP}{TP + FN}$$

The important aspect of both these (and most other) metrics is that they are based on relevance. In the case of this project, relevance can not be precisely measured, since the system was not run in a real-life situation. The developer can not know if the user would actually click on the company, or if the company truly is a potential customer. These metrics should be monitored, but the provided data is not enough to calculate them.

5.1.2 Possible improvements

More data is necessary to measure the quality of the recommender system. This data can only be obtained by running the algorithm with real users. Furthermore, better data about the user interaction is crucial, a single click does not provide much added value. How long do the users stay on the company detail page, what part of the page are they looking at, what links do they click at, etc. More information brings better results.

To further improve the algorithm, the data about client organizations could be used. A profile vector for an organization could be created and combined with all the users who belong to that organization. The vector would then be continuously updated based on companies that the users interact with. That would bring benefits of collaborative filtering, upgrading the system to a hybrid one.

A technical improvement to increase the effectiveness of the program would be to use multiple threads while calculating the scores. Asynchronous threading could make the process considerably faster, especially in the future, as the volume of data increases over time with new companies, new users, and more interactions.

5.1.3 Comparison to other solutions

Comparing the system to other currently running solutions is problematic since the systems in use (or even previous versions) are usually not public. The filtering algorithms of Netflix, Instagram, or Amazon are most probably significantly more complex, processing big data, and combining several architectural approaches at the same time.

For example, the recommender system used by Amazon today for a large number of products, including movies, is a multi-layered neural network. But just 20 years ago, Amazon was using an item-to-item collaborative approach to recommend products on their e-shop. (22) The main idea of this approach might be similar to the one implemented in this thesis - using user history to recommend similar items based on their attributes. However, the specific implementation was probably more complex and processed vastly more data.

5.2 Interface

The goal of the interface was to create a clear and simple environment for the developer to test the system and quickly test the results. The resulting website meets these requirements, as it offers a catalog with companies where the developer can log in as any real user and try to interact with companies. The interactions are saved into the database and then used as input when calculating user profiles.

The second part of the website is a console for algorithm configuration. The developer can set up all the important aspects of the calculations here without changing and recompiling the source code.

5.2.1 Possible improvements

The interface could be improved in several places. The catalog page could include a table with all the companies, a search bar or sorting options could be implemented. The website is now used only locally, but if it were to be put online, there are several security concerns to be taken care of. A log-in page would have to be created, allowing only the entrusted users to enter, which would also mean implementing a whole credentials system.

Conclusion

Results

The result of this thesis is a functional recommender system that proposes potential customers to sales representatives in B2B. The recommender was designed using a content-based approach and basic mathematical methods of linear algebra. The system successfully processed data from a local database, that was created in order to store the input dataset as well as new user interactions, and results of the algorithm.

Another part of the system is a web interface created in the Python-based library Django. This interface serves as a convenient way to configure the algorithm, run it, and see the results. The interface also allows to log in as a real user and interact with companies. The goals of this thesis have been met as the resulting system is functional and designed in a way to be used in a real use case. All the functional requirements has been met. The methods and technologies used are modern and meet the current coding and architecture standards.

The thesis describes how the system was designed and implemented. Furthermore, it describes the history of recommender systems and mentions some of the successful existing solutions. The current approaches to designing such systems are also briefly described, including the pros and cons of each one. One of these approaches is then chosen and used to design a new system from scratch. The whole process is described in detail.

Future steps

After the algorithm is used in the real use case, there will be more data to determine how precise it is. Based on this data, further improvements will most likely be done. The algorithm is now based strictly on the company contents, one of the probable improvements is using other users to improve the recommending algorithm. Either users that belong to the same client organization, or users that behave in a similar way.

The administrative interface will further serve as a platform for developers to test and configure the system. Although the catalog might get exchanged for a development version of the real user interface. The part of the configuration page could be used separately, or it could also become a part of the development version of the real website.

References

1. FALK, Kim. *Practical Recommender Systems*. Manning Publications Co., 2019. ISBN 9781617292705.
2. RICCI, Francesco; ROKACH, Lior; SHAPIRA, Bracha. *Introduction to Recommender Systems Handbook*. Boston, MA: Springer, 2011. ISBN 978-0-387-85819-7.
3. BLANDA, Stephanie. Online Recommender Systems – How Does a Website Know What I Want? 2015. Available also from: <https://blogs.ams.org/mathgradblog/2015/05/25/online-recommender-systems-website-want/>.
4. KARLGREN, Jussi. *An Algebra for Recommendations*. Stockholm University, 1990.
5. KARLGREN, Jussi. A digital bookshelf: original work on recommender systems. 2017. Available also from: <https://jussikarlgren.wordpress.com/2017/10/01/a-digital-bookshelf-original-work-on-recommender-systems/>.
6. COHEN, Jacques (ed.). *Commun. ACM*. 1992, vol. 35, no. 12. ISSN 0001-0782.
7. RESNICK, Paul. *In Proceedings of the 1994 ACM conference on Computer supported cooperative work*. GroupLens: an open architecture for collaborative filtering of netnews. ACM, 1994.
8. GROUPLENS. *Projects*. 2021. Available also from: <https://grouplens.org/about/projects/>.
9. PRIZE, Netflix. 2009. Available also from: <https://www.netflixprize.com>.
10. VILLARROEL, J.A.; TAYLOR, J.E.; TUCCI, C.L. Innovation and learning performance implications of free revealing and knowledge brokering in competing communities: insights from the Netflix Prize challenge. *Comput Math Organ Theory*. 2013, no. 19, pp. 42–77.
11. SINGH, Pradeep; DUTTA PRAMANIK, Pijush; DEY, Avick; CHOUDHURY, Prasennjit. Recommender Systems: An Overview, Research Trends, and Future Directions. *International Journal of Business and Systems Research*. 2021, vol. 15, pp. 14–52.
12. RIEL, John; KONSTAN, A. Joseph. *Word of Mouse: The Marketing Power of Collaborative Filtering*. Business Plus, 2002. ISBN 978-0-7595-2727-0.
13. TAN, Xinyue. *Collaborative Filtering Recommender Systems*. 2019. Available also from: <https://github.com/XinyueTan/Collaborative-filtering-recommender-systems>.
14. FOUNDATION, The Django Software. 2021. Available also from: <https://docs.djangoproject.com/en/3.1/>.
15. FOUNDATION, Django Software. *URL Dispatcher*. 2021. Available also from: <https://docs.djangoproject.com/en/3.2/topics/http/urls/>.
16. ROSSUM, Guido van; WARSAW, Barry; COGHLAN, Nick. *Style Guide for Python Code*. 2001. Available also from: <https://www.python.org/dev/peps/pep-0008/>.

17. CONSORTIUM, SQLite. 2021. Available also from: <https://www.sqlite.org/about.html>.
18. FOUNDATION, Python Software. *CSV File Reading and Writing*. 2021. Available also from: <https://docs.python.org/3/library/csv.html>.
19. HENRY MALTBY, Hobart Pao; KHIM, Jimin. *Linear Transformations*. 2021. Available also from: <https://brilliant.org/wiki/linear-transformations/>.
20. HARRIS, Charles R.; MILLMAN, K. Jarrod; WALT, Stéfan J. van der; GOMMERS, Ralf; VIRTANEN, Pauli; COURNAPEAU, David; WIESER, Eric; TAYLOR, Julian; BERG, Sebastian; SMITH, Nathaniel J.; KERN, Robert; PICUS, Matti; HOYER, Stephan; KERKWIJK, Marten H. van; BRETT, Matthew; HALDANE, Allan; RÍO, Jaime Fernández del; WIEBE, Mark; PETERSON, Pearu; GÉRARD-MARCHANT, Pierre; SHEPPARD, Kevin; REDDY, Tyler; WECKESSER, Warren; ABBASI, Hameer; GOHLKE, Christoph; OLIPHANT, Travis E. Array programming with NumPy. *Nature*. 2020, vol. 585, no. 7825, pp. 357–362. Available from DOI: 10.1038/s41586-020-2649-2.
21. BONDARENKO, Kirill. *Precision and recall in recommender systems. And some metrics stuff*. 2019. Available also from: <https://bond-kirill-alexandrovich.medium.com/precision-and-recall-in-recommender-systems-and-some-metrics-stuff-ca2ad385c5f8>.
22. HARDESTY, Larry. *The history of Amazon's recommendation algorithm*. 2019. Available also from: <https://www.amazon.science/the-history-of-amazons-recommendation-algorithm>.

Attachments

A. Data and code

Attachment A: Provided dataset *UsageDataset_Anonymized.xlsx*

Attachment B: Project source code *recommender_web.zip*